

Quick audit: ledger-service (sample deliverable)

Client: Larkspur Pay, a fictional payments company. **Service reviewed:** `ledger-service`, the stage that applies validated payments to double-entry postings in Postgres and publishes each outcome.

This is a sample of the Starter tier: a configuration and code review of a single service. **No load test was run for this document**, so it contains no measured performance figures; configuration values quoted below are read from the service's committed configuration files at generation time. The client, the service and all data are fictional. The benchmark evidence for the whole pipeline is in `AUDIT_REPORT_SAMPLE.md`.

Scope and method

Read the service's code and configuration as found (the baseline profile), traced one payment through it, and looked for cost that grows with volume, contention that limits concurrency, and failure behaviour that loses or stalls work. Correctness properties (idempotent application, non-negative balances, per-debtor ordering) were reviewed first and are covered by tests; they are not the subject of the findings below.

Findings

The review found 7 findings, ordered by expected value. Where a finding corresponds to one in the full report, its identifier is given in brackets; the others are specific to this service.

Q-A (F-04) One transaction and many round trips per record

- **Observation.** Each record opens a transaction, locks the debtor, creditor and settlement account rows, checks for a duplicate, reads the day's outflow, inserts the ledger legs (debtor to settlement to creditor), updates the balances and writes an outbox row, one statement at a time.
- **Impact.** The number of commits and network round trips grows one-for-one with payments, and the account row locks are held across all of them. Expected to be the largest single cost in this service; not confirmed by a measurement.
- **Recommendation.** Process a poll as one transaction with batched statements, lock the accounts of the whole batch in one ordered statement, and keep the dedup record in the same transaction. Isolate a poison record instead of failing the batch.
- **Effort. M. Risk.** High: the dedup, sequence-ordering and lock-ordering rules must be preserved exactly.

Q-B (F-06) The daily-limit query has no supporting index

- **Observation.** Every payment sums the debtor's postings for the current day. The postings table has only its primary key, so the query reads the whole table.
- **Impact.** Cost per payment grows with the size of the table, so the service slows down as data accumulates, independent of load.
- **Recommendation.** Index postings by account and creation time. Alternatively keep a running per-account daily total, which trades a write for the read.
- **Effort.** S. **Risk.** Low.

Q-C (F-03, F-11, F-12) Concurrency, partitions and one settlement row cap parallelism

- **Observation.** The service runs 1 consumer thread over 3 partitions, and records are keyed by merchant rather than by debtor account. Every payment also locks the same single settlement account, so once consumer threads are raised they queue on that one row.
- **Impact.** Applies one payment at a time regardless of hardware, and a large merchant's records share one partition. Raising the thread count alone helps only once partitions and database connections allow it.
- **Recommendation.** Key by debtor account, size partitions from measured per-partition throughput, then raise consumer threads to match. Per-debtor ordering is enforced by sequence numbers, so the key change does not put ordering at risk.
- **Effort.** M. **Risk.** Medium: real parallelism exercises the locking (one latent lock-ordering bug was found while doing this).

Q-D (F-07) Connection pool left at its default

- **Observation.** The database pool uses the driver default size. Each consumer thread needs a connection for the duration of its unit of work, so the pool matters as soon as the thread count is raised.
- **Impact.** Once concurrency is raised, threads wait for connections; sized badly, the pool becomes the limit rather than the database.
- **Recommendation.** Size the pool from the database's capacity shared across all services, and set the acquisition timeout above the longest unit of work. Measure rather than guess.
- **Effort.** S. **Risk.** Low.

Q-E The outbox is flushed from the consumer thread

- **Observation.** After each record the listener calls a synchronised flush that sends outbox rows to the broker and waits for each acknowledgement.
- **Impact.** With more than one consumer thread the synchronisation serialises them at the flush, and each record's latency includes broker round trips for rows it did not write.
- **Recommendation.** The outbox already has a scheduled publisher; stop calling the flush from the listener and let the scheduled publisher do the work, batch the sends asynchronously, and delete only acknowledged rows. The outbox itself is the right design and should stay.

- **Effort.** S. **Risk.** Low.

Q-F Dead-lettering can strand a debtor

- **Observation.** After bounded retries a failing record is dead-lettered. Later payments for the same debtor are held until the missing sequence number arrives, and nothing consumes or alerts on the dead-letter topic.
- **Impact.** One poison message can stall every later payment for its debtor indefinitely, without an alarm.
- **Recommendation.** Alert on the dead-letter topics, provide a re-drive procedure, and define how a sequence gap is closed. Retry transient database errors for as long as they last instead of dead-lettering them.
- **Effort.** M. **Risk.** Medium.

Q-G (F-09) Metrics count attempts, and logs carry full payloads

- **Observation.** Applied and parked counters are incremented inside the transaction, so a retry or redelivery counts again. Every event is also logged at info level with its full payload, and a new JSON mapper is built for each message.
- **Impact.** Rates on a dashboard can overstate throughput after failures; a report built from them would be wrong. Full payloads put account identifiers and amounts into the logs, and the per-message mapper and log volume cost CPU on the hot path.
- **Recommendation.** Treat the counters as operational signals only; derive business figures from the database, and increment after commit if precise counts are needed. Share one mapper and log identifiers at debug level, not payloads at info.
- **Effort.** S. **Risk.** Low.

Suggested order

Start with Q-B and Q-D (small, safe), then Q-E, then Q-C together with Q-A once partitions and pools allow real concurrency. Treat Q-F as parallel work: it is about operability rather than speed. Q-G can be done at any time.

What a full audit would add

A load test with the service-level objective, consumer-lag and database measurements per stage, an ablation that measures each change on its own, and the neighbouring services. The full sample (`AUDIT_REPORT_SAMPLE.md`) shows that output.